UNIVERSIDADE DA CORUÑA

Aprendizaje Automático a Gran Escala

Grado en Ciencia e Ingeniería de Datos

Práctica 2.1: Aprendizaje Federado (2026-2027)

1. Descripción e instrucciones generales

La Práctica 2 tiene dos partes que se entregan juntas: la **2.1, aprendizaje federado** (este enunciado) y la **2.2, aprendizaje continuo con River** (enunciado aparte). Las instrucciones de esta sección son comunes a las dos.

En la 2.1 vais a implementar **desde cero, en PyTorch y sin librerías de aprendizaje federado**, un sistema federado simulado en un solo ordenador: el reparto de los datos entre clientes, el entrenamiento local, la agregación en el servidor y la medida del coste de comunicación. Después lo usaréis para estudiar cómo afectan la heterogeneidad de los datos y los hiperparámetros propios del aprendizaje federado.

- El trabajo se realiza en **grupos de dos o tres** estudiantes.
- Se entrega un **.zip** con el código de las dos partes, un **README.md** con instrucciones de ejecución y una **memoria en PDF**.
- Se permite usar asistentes basados en LLM, siempre que la memoria incluya una sección que explique **para qué y cómo** se han usado. Sois responsables de todo el código que entregáis y de su calidad.
- Fecha límite: **11 de diciembre de 2026, a las 23:55**.

2. Código base

El código base está en `https://github.com/AlejandroDopico2/aage-fl`. Ya resuelve la parte que no es aprendizaje federado: la carga de Fashion-MNIST, la configuración, el registro de resultados y las gráficas. El `README.md` del repositorio explica la instalación (CPU, `pip`, y Google Colab) y hay una guía rápida de PyTorch relacionada con Keras.

Los fragmentos que tenéis que programar lanzan `NotImplementedError("TODO: ...")` y su *docstring* explica qué deben hacer. Este es el orden recomendado:

#	Archivo	Qué hay que implementar
1	`fl/data.py`	`iid`, `dirichlet`, `dirichlet_shards`
2	`fl/models.py`	`MLP` (el de las diapositivas) y `CNN`
3	`fl/codecs/base.py`	`state_dict_nbytes`
4	`fl/server.py`	`sample_clients` y `Server.run_round`
5	`fl/aggregators/fedavg.py`	`FedAvg.aggregate`
6	`fl/clients/base.py`	`Client.local_train`
7	`fl/clients/fedprox.py`	`FedProxClient.loss`
8	`fl/server.py`	`LocalOnly.run_round`

Con los seis primeros ya se puede lanzar un experimento completo. Cada hueco tiene un test público (`pytest`); al corregir se usarán además tests adicionales. **No cambiéis las cabeceras de las funciones con hueco ni el formato de** `metrics.csv`.

Los experimentos se lanzan **sin tocar el código**. Toda ejecución parte de `configs/default.yaml`, y lo que cambia se indica de una de estas dos formas:

- desde la línea de comandos, con `--set clave=valor`;
- con un fichero YAML que solo contiene lo que cambia (hay ejemplos en `configs/examples/`).

Cada ejecución guarda su configuración, sus métricas por ronda y el histograma del reparto en `runs/<nombre>/`, y `plot.py` compara varias ejecuciones:

```
python run.py --set partition.alpha=1.0 client.local_epochs=5 server.fraction=1.0
python run.py --config configs/examples/fedprox.yaml
python plot.py runs/* -o comparativa.png
```

3. Tareas

3.1. Escenario federado

- 10 clientes. El modelo global se evalúa en cada ronda sobre el conjunto de test completo de Fashion-MNIST.
- Tres repartos de los datos: **IID**, **Dirichlet(α)** y k **clases por cliente** (con proporciones Dirichlet entre ellas).
- Para cada reparto usado, la memoria debe incluir el **histograma de clases por cliente** (`partition.png`).

3.2. Modelos

- **MLP** de las diapositivas: $784 \to 200 \to 200 \to 10$, con ReLU entre capas.
- **CNN** de diseño libre con un máximo de **100.000 parámetros**. Justificad el diseño.

3.3. Entrenamiento federado

- Muestreo de una fracción C de los clientes en cada ronda y la ronda completa: envío del modelo global, entrenamiento local durante E épocas, subida y agregación.
- **FedAvg**: media de los pesos ponderada por el número de ejemplos de cada cliente.
- **FedProx**: el entrenamiento local añade el término proximal $\frac{\mu}{2}\|w - w_t\|^2$ a la pérdida.

3.4. Comunicación

- Medid en cada ronda los **bytes de bajada y de subida**.
- Implementad una **cuantización a 8 bits** de lo que suben los clientes. Decidís dónde y cómo implementarla. Debe poder activarse con `--set server.codec=int8` (hay un test público que lo comprueba).

3.5. Referencias

- **Centralizado** (techo): `configs/examples/centralized.yaml`, un único cliente con todos los datos.
- **Solo local** (suelo): cada cliente entrena únicamente con sus datos (`LocalOnly`).

4. Experimentos y preguntas de análisis

Salvo que se indique otra cosa, usad la configuración por defecto (MLP, Dirichlet $\alpha = 0{,}1$, $C = 0{,}5$, $E = 1$, 30 rondas). Las diferencias entre métodos pueden ser del orden del ruido entre ejecuciones: **repetid cada comparación con al menos 3 semillas** y dad la media y la desviación típica.

1. **Heterogeneidad**: IID, $\alpha \in \{1;\ 0{,}1\}$ y $k = 2$ clases por cliente, junto con las dos referencias. ¿Cómo cambian la accuracy final y la estabilidad de las curvas? ¿Por qué?
2. **Épocas locales**: $E \in \{1, 5\}$ con datos IID y no-IID. ¿Cómo afecta el número de épocas locales?
3. **Participación**: $C \in \{0{,}3;\ 0{,}5;\ 1{,}0\}$. Relacionad la accuracy con los bytes transmitidos.
4. **FedAvg frente a FedProx**: al menos dos valores de μ. ¿Cómo afecta FedProx según el escenario? Explicad por qué.
5. **Comunicación**: accuracy frente a MB transmitidos, con y sin cuantización int8. ¿Cuánto se ahorra y cuánto se pierde?
6. **CNN**: repetid el experimento 1 con vuestra CNN y comparad con el MLP.

5. Extensiones opcionales

- Otras formas de comprimir la comunicación (por ejemplo, enviar solo las componentes más grandes de la actualización).
- Otros agregadores o variantes del entrenamiento local, siguiendo la estructura del código (una clase en su carpeta, registrada en su `build_*`).
- Accuracy por cliente, o más clientes.

En unas dos semanas, junto con la parte 2.2, se presentará un reto adicional de aprendizaje federado.

6. Memoria y evaluación

La memoria debe describir las decisiones de implementación, incluir los histogramas de los repartos y las gráficas de cada experimento, y responder a las preguntas de la sección 4 razonando a partir de los resultados.